www.ambarella.com

SDK8_AN_Security

Summary Description

This document explains the application note for security and it's suitable for Ambarella CV Chip family.

Confidential
For Mekotang only
(2025-05-05)

Copyright Notice

Copyright © 2023 Ambarella International LP.

The contents of this document are proprietary and confidential information of Ambarella International LP.

The material in this document is for information only. Ambarella assumes no responsibility for errors or omissions and reserves the right to change, without notice, product specifications, operating characteristics, packaging, ordering, etc. Ambarella assumes no liability for damage resulting from the use of information contained in this document. All brands, product names, and company names are trademarks of their respective owners.

For complete Ambarella contact information, please visit www.ambarella.com.

Table of Contents

Contents

1 Concept and Background 4
1.1 Abbreviation 4
1.2 Root of Trust 5
1.2.1 Reference Approach 5
1.3 Chain of Trust 6
1.3.1 Reference Approach 7
1.4 Arm Trustzone 7
1.4.1 Reference Approach 9
1.5 Anti-rollback Attack 9
1.5.1 Reference Approach 9
1.6 Anti-Hardware Clone 10
1.6.1 Reference Approach 10
1.7 Debugger Protection 10
1.7.1 Reference Approach 10
1.8 Memory Scrambling 10
1.8.1 Reference Approach 10
1.9 DMA Attack 11
1.9.1 Reference Approach 11
1.10 Other Security Measures 11
1.10.1 Boot-to-USB Protection 11
1.10.2 Misc Hardware Modules 11
2 Booting 12
2.1 Boot Options 12
2.2 Boot Flow: Single OS 13
2.3 Boot Flow: Ambalink Dual-OS 15
2.3.1 Resource Assignment 17
2.3.2 Image Load Sequence 17
2.3.3 Trust Sequence 17
2.3.4 IPC 17

Confidential
For Mekra Lang only
(2025-05-19)

3 Configuration Setup 19
3.1 Generate RSA Key 19
3.2 Generate Ed25519 Key 19
3.3 OP-TEE 20
3.4 OTP 21
3.4.1 OTP Caution 27
4 iCam SVC Secure Boot 27
4.1 How to Generate and Burn OTP Public Key (CV5x) 27
4.2 How to Generate and Burn OTP Public Key (non-CV5) 29
4.3 Compile Secure Boot SVC 30
4.3.1 CV22 30
4.3.2 CV5 31
4.4 Implementation with mbedtls FOSS Library 34
4.5 Build Secure Boot Linux Kernel (non-CV5) 35
4.6 Build Secure Boot DSP uCode and FlexiBin in SECURE Partition 35
5 Verification 37
5.1 Development Stage 37
5.2 Mass Production (MP) Stage 38
5.3 Return Merchandise Authorization (RMA) Stage 38
6 Revision History 40

Confidential
For Mekra Lang only
2-24-(1105-2025)

1 Concept and Background

This chapter introduces the security concept and the background knowledge.

1.1 Abbreviation

Abbreviation	Description
BLD	Bootloader binary firmware
BST	Bootstrap binary firmware
SYS	SYSTEM (RTOS, Application) binary firmware
ATF	Arm trusted firmware
BL	Boot loader stage in ATF
EL	Exception level
AES	Advanced encryption standard
RSA	An asymmetric public-private key cryptosystem named after three inventors
EdDSA	An asymmetric public-private key cryptosystem that is found to be more secure than RSA.
Ed25519	Ed25519 is the EdDSA signature scheme using SHA-512 and Curve25519
SHA	Secure hash algorithm
OTP ROM	One time programmable Read-Only Memory
ROTPK	Root of trust public key
HUK	Hardware unique key
GP	Global platform

Confidential For Meera Lang only 2.2 (1652023)

CA	Client application
TA	Trusted application
PTA	Pseudo TA
SDK	Ambarella Software Development Kit
FOSS	Free or Open Source Software
SVC	Reference Application in Ambarella SDK
POC	Power On Configuration for SoC to latch the initial hardware setting
ThreadX	A Commercial Real-Time Operating System
TEE	Trusted Execution Environment
OP-TEE	Open Portable Trusted Execution Environment
QNX	A Commercial POSIX-compliant Real-Time Operating System
IPL	Initial Program Loader of QNX

Confidential
For Mekra Lang only
2.2 / 1053025 /

1.2 Root of Trust

Root of Trust (RoT) is a source that can always be trusted within a cryptographic system. Because cryptographic security is dependent on keys to encrypt and decrypt data as well as perform functions such as generating digital signatures and verifying signatures, RoT schemes generally include a hardened hardware module.

1.2.1 Reference Approach

Ambarella considers Secure ROM code and OTP ROM (Public Key) as the Root of Trust. The Secure ROM code is physically hardwired and not changeable. The digital signature algorithm in Secure ROM code is RSA (CV2x/CV2xfs/H32) or Ed25519 (CV5x). The OTP ROM is One-Time-Programmable ROM device which can be programmed once only. Please refer to OTP chapter for details.

1.3 Chain of Trust

For security, multiple certificates are sometimes required to verify various aspects of components, making management of different public keys in each certificate difficult. To help manage the certificates and ensure security, it is important to first confirm that the root certificate is secure. If the root certificate has not been compromised, it can be used to verify the subsequent certificates and establish a chain of trust. Here is an example figure from wiki page.

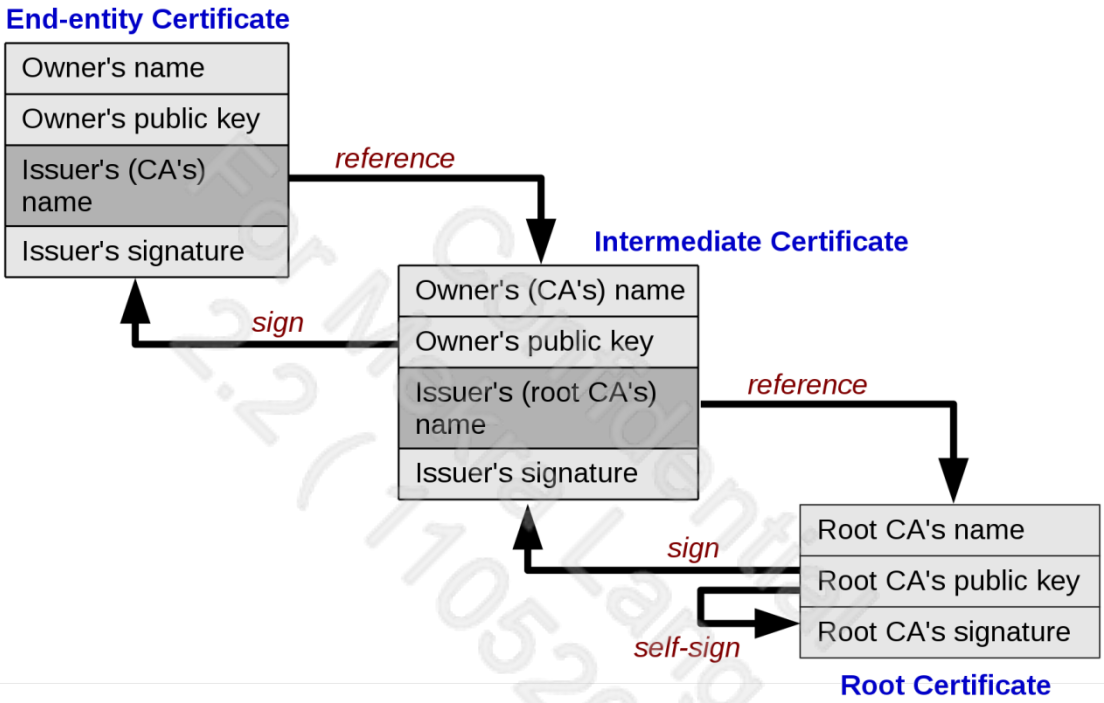


Figure 1 *Chain of Trust.*

The Chain of Trust is used in boot protection. During the secure boot flow, if a signature check fails at any stage, the entire boot flow fails.

Secure boot adds the cryptographic check to each stage of the secure world boot process. This process ensures the integrity of all the secure world software images that are executed, preventing any unauthorized or maliciously modified software from running. The secure boot flow is divided into several boot procedures, for example, secure ROM, boot strap, Arm trusted firmware, boot loader, and OS. To implement the secure boot and prevent unauthorized access, the first procedure should be initiated from ROM, instead of RAM. All subsequent procedures are responsible for checking the integrity of the next procedure, and thereby, forming a chain of trust until all procedures have completed.

The Secure ROM ensures that the first boot stage code is read-only. The public key programmed in the OTP is also read-only. Each boot procedure checks the signature at each stage, in a chain-like method, ensuring the integrity of each boot procedure.

1.3.1 Reference Approach

Ambarella Secure ROM reads the public key from the OTP and uses the key to check the integrity of the BST binary. Then, the BST binary uses the public key (or derived public key from this public key) to check the integrity of the next image and form the chain of trust. It is implemented in Ambarella BST code and native ATF open-source code.

1.4 Arm Trustzone

The Arm TrustZone is the hardware-based security built into SoC's by semiconductor chip designers who want to provide secure end points and roots of trust. Arm TrustZone implements the trusted execution environment (TEE). Arm TrustZone creates an isolated secure world which can be used to provide confidentiality and integrity to the system. The security of the system is established by partitioning all the SoC's hardware and software resources, so that they exist in one of the two worlds: the secure world for the security subsystem, or the normal world for all else. For further details, please visit the website: https://developer.arm.com/technologies/trustzone.

Arm trusted firmware (ATF) is open-source secure world software for the Arm Cortex, which includes secure monitor executing at Exception Level 3 (EL3). For details on the Arm trusted firmware, please visit the website: https://github.com/ARM-software/arm-trusted-firmware.

The open-portable trusted execution environment (OP-TEE) in Linux is an open source environment which uses Arm TrustZone technology. This component meets the GlobalPlatform TEE system architecture specification. For further details, please visit the website: https://www.op-tee.org/.

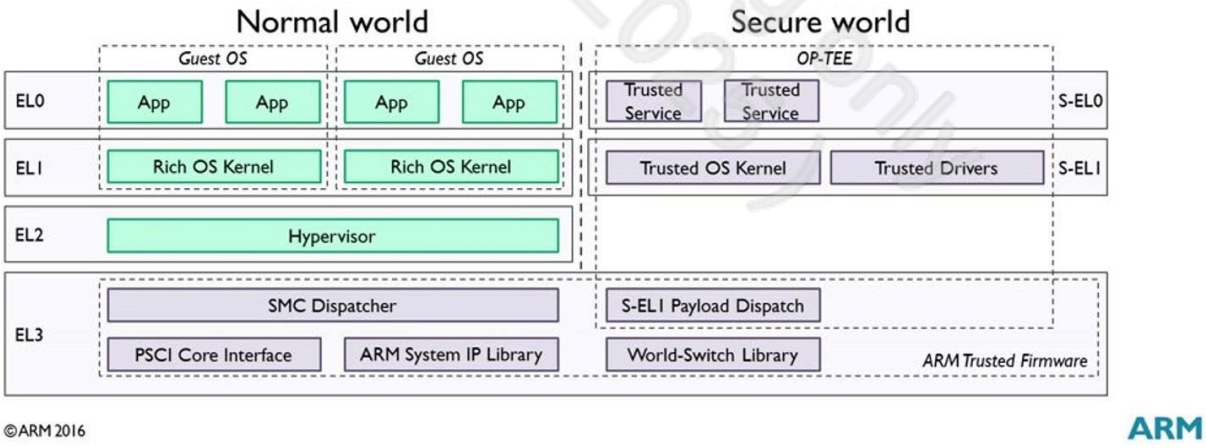


Figure 2 Arm Trustzone.

There are different Exception Levels, EL3 runs Secure Monitor, EL2 runs hypervisor, EL1 runs OS Kernel and EL0 runs user application.

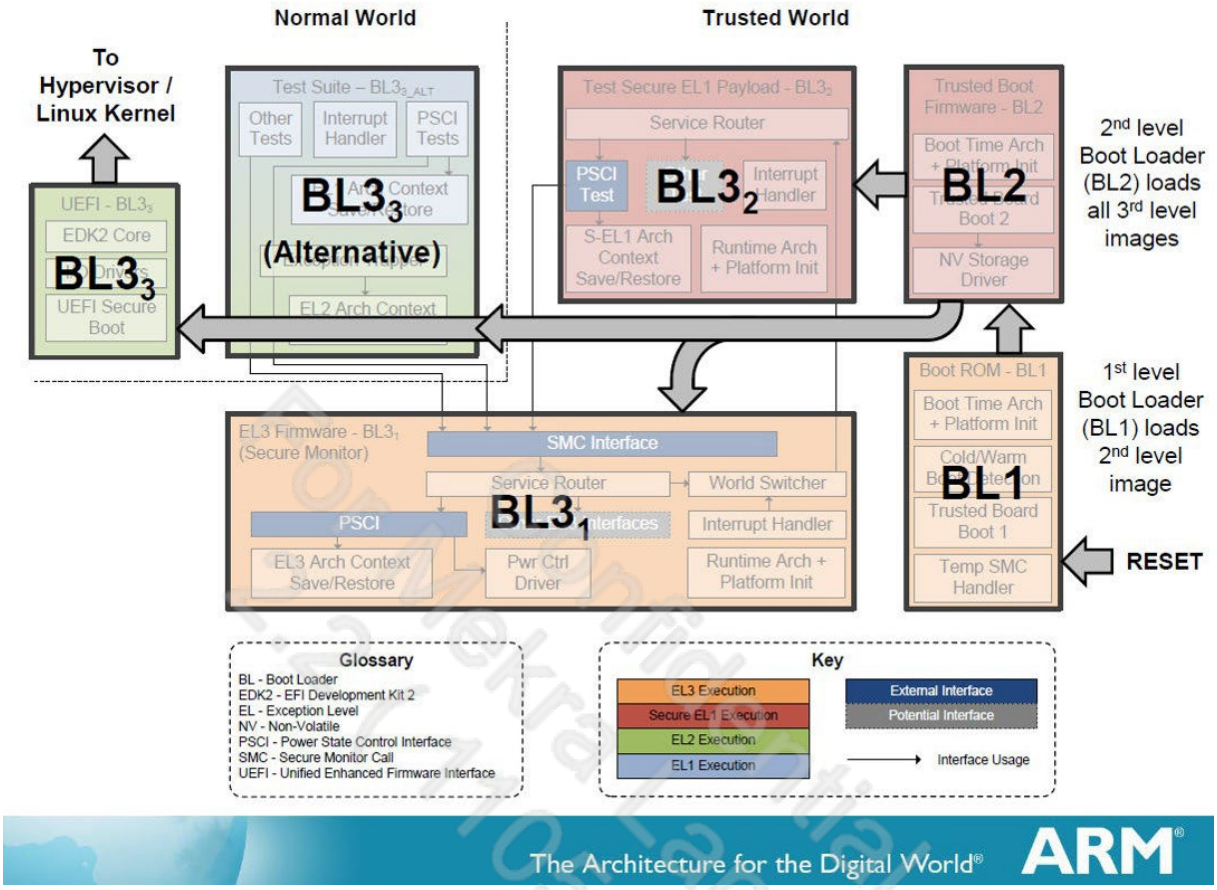


Figure 3 *Arm Boot Loader Stages.*

The boot process is divided into several Boot Loader stages, BL1 for BST, BL2 for the Loader of BL3 images, BL31 for Arm Trusted Firmware (open-source Secure Monitor), BL32 for TEE OS, and BL33 for Normal World Boot Loader.

The following diagram shows how the OP-TEE functions with ATF and Linux.

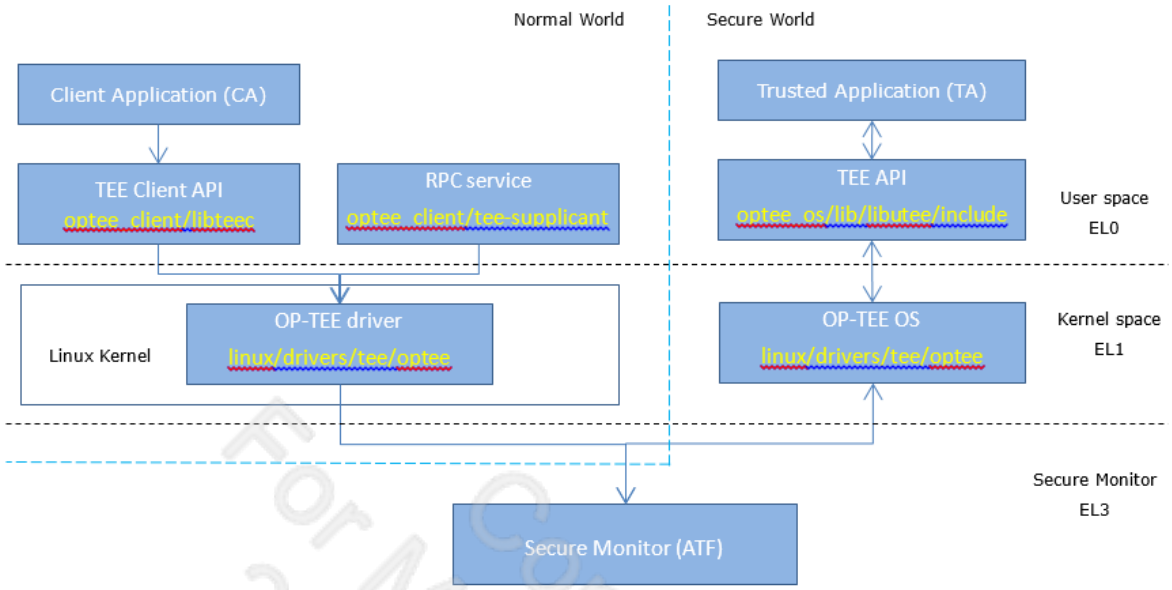


Figure 4 OP-TEE System Diagram.

The trusted OS is accessible from the Rich OS (Linux) using the GlobalPlatform TEE client API specification, which also is used to trigger secure execution of applications within the TEE. OP-TEE contains the complete stack from the normal world client APIs (optee_client), the Linux kernel TEE driver (optee_linuxdriver), the trusted OS (optee_os) + the secure monitor (atf) and the test suite (optee_test).

1.4.1 Reference Approach

Ambarella CPU (Arm Cortex) is protected by Arm Trustzone hardware feature. During booting, each ARM core can be configured as secure or non-secure world. CPU core running in the non-secure world cannot be changed to the secure world by itself. Normal world cannot access the hardware and software resources in the secure world. This can prevent the secure memory region data from being accessed by non-secure world ARM core. In Ambarella Dual-OS SDK, there is no EL2 hypervisor but there is an extra RTOS (when Dual-OS is enabled) running in secure world.

1.5 Anti-rollback Attack

To bypass the security boot, an attacker might attempt to reprogram the previous vulnerable firmware.

1.5.1 Reference Approach

Ambarella OTP is used for the monotonic counter purpose. A reference application firmware update flow can advance the monotonic counter bit-by-bit for each firmware update. Application can call OTP service API via SMC or OP-TEE to read out the fields on OTP ROM for the current firmware version and update the blank bit to advance the counter by OTP unlocking and writing API. Therefore, checking the monotonic counter in firmware

update procedure and in boot procedure could help prevent an anti-rollback attack. Please refer to OTP chapter for more details.

1.6 Anti-Hardware Clone

There are two types of anti-hardware clones: hardware copy clones including NAND, and clones with manufacturing software. To prevent hardware copy clones including NAND, manufacturers can equip the firmware of each device with a unique encryption key, though clones with manufacturing software represent a more powerful and difficult security problem.

1.6.1 Reference Approach

Ambarella activates each device (using a private key to assign a unique ID for each chip) during mass production, preventing an attacker without the private key from generating the correct digital signature on a cloned device. Please refer to the OTP chapter for more details.

1.7 Debugger Protection

JTAG (IEEE1149.x) interface can be used by attackers to break the security of the device.

1.7.1 Reference Approach

JTAG (IEEE1149.x) interface can be disabled by the hardware during Ambarella secure booting. In CV2x/CV2xfs/H32, it is automatically disabled during secure boot. In CV5x, it needs to be disabled specifically by programming the corresponding OTP bit.

Customers can build an authentication software flow for external user/application to communicate with the secure OS in order to authenticate the use of JTAG interface. For example, the secure OS can have a challenge-response protocol via CAN or UART to authorize the JTAG to enable requests from external users. After authentication, JTAG interface can be re-enabled at runtime. This is done by setting Ambarella scratchpad register ***AHBSP_SEC_JTAG_EN_REG*** from 0 to 1 and can only be set by secure CPU.

1.8 Memory Scrambling

Memory Scrambling is supposed to prevent forensic and reverse-engineering analysis based on DRAM data remanence by effectively rendering various types of cold boot attacks ineffective. This feature turns user data written from the main memory into pseudo-random patterns.

1.8.1 Reference Approach

Ambarella DRAM controller has a built-in AES encrypt/decrypt engine for memory scrambling. Once the system is secure booted via POC or eFuse, the bootstrap code will generate an AES key by configuring the internal TRNG module and store the key into a write-only DRAM register before enabling the DRAM controller. After

the DRAM controller is enabled, all memory data will go through the AES engine and be encrypted before they are saved on DRAM. The encrypted data will be decrypted when reading out from DRAM via DRAM controller.

1.9 DMA Attack

DMA attack is a type of side channel attack in computer security, in which an attacker can penetrate a computer or other device by exploiting the presence of the high-speed expansion ports that permit direct memory access (DMA). The attack is done by using direct hardware access to read or write directly to the main memory without any operating system supervision or interaction.

1.9.1 Reference Approach

ATT (Address Translation Table) of DRAM controller can limit the access range to protect DRAM content. When the DRAM controller receives a request from a client, it will translate the virtual address to physical address according to this table. This prevents the memory content from being tampered by DRAM client via DMA transaction. For details, please refer to ***struct AMBA_DRAM_ATT_INFO_s*** and ***struct AMBA_DRAM_ATT_CLIENT_INFO_s*** in Ambarella SDK.

1.10 Other Security Measures

The following section lists some other security measures in Ambarella platform.

1.10.1 Boot-to-USB Protection

The USB boot function can be disabled in Ambarella SoC. USB is automatically disabled during secure boot. In CV5x, it needs to be disabled specifically by programming the corresponding OTP bit.

This prevents the device from being controlled, for example, re-programming the firmware, by the USB host.

1.10.2 Misc Hardware Modules

Ambarella has hardware TRNG (True Random Number Generator), which is a module, rather than an algorithm, that generates random numbers from a physical process (thermal noise). The TRNG AmbaRNG_DataGet() API can be used to get TRNG random numbers.

Ambarella Arm Cortex CPU supports the ARMv8 cryptography extensions. The cryptography extension adds new A64, A32, and T32 instructions to an advanced SIMD that accelerates the following:

- Advanced encryption standard (AES) encryption and decryption.
- The secure hash algorithm (SHA) functions SHA-1, SHA-224, and SHA-256.
- Finite field arithmetic used in algorithms such as Galois / counter mode and elliptic curve cryptography.

The hardware acceleration code is available in open-source library named openssl.

2 Booting

This chapter introduces the boot option and explains the boot flow.

2.1 Boot Options

In Ambarella Chips, some boot storage devices are available for selection.

Boot Interface	Boot device	Support Secure Boot	Purpose
Parallel NAND	8 bits data NAND Flash	Yes	Boot
SPI NAND	1~4 bits data NAND Flash	Yes	Boot
SPI NOR	1~4 bits data NOR Flash	Yes	Boot
EMMC	1~8 bits EMMC	Yes	Boot
USB	To PC via USB port	No	Development

Table 1 Boot Devices.

As listed in the table above, Ambarella chips support several different device boot options. Each boot option is exclusive with the other. The selection of boot option is either via Power-On-Configuration on board or pre-burned effuse inside the chip. If the Secure Boot function is enabled, the USB boot function will be disabled.

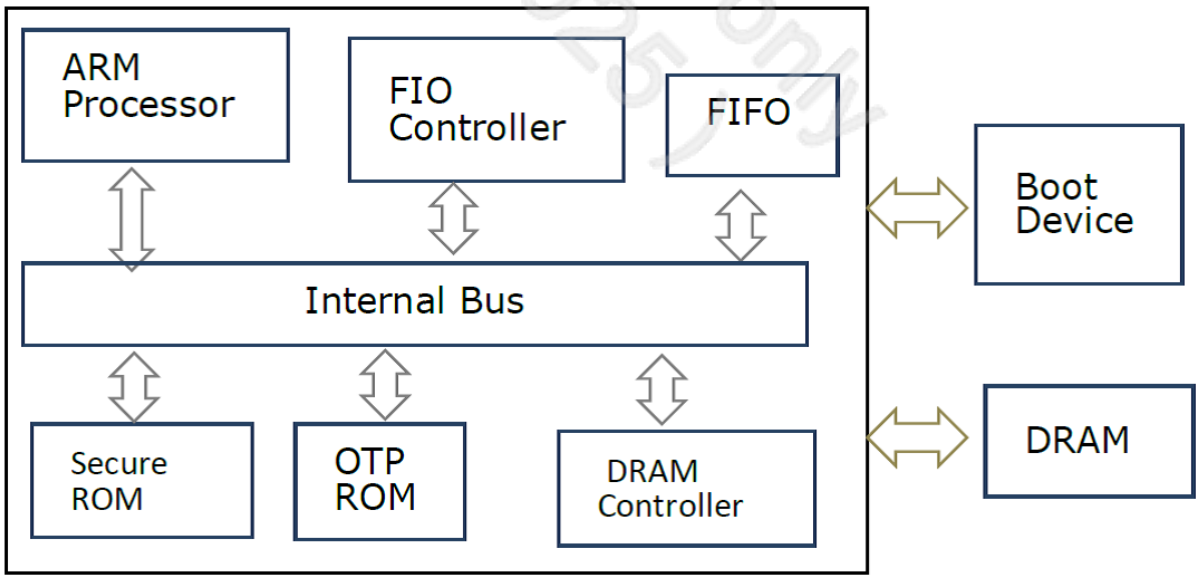


Figure 5 Simplified System Blocks.

When a chip is released from the reset state, BST (Boot Strap) firmware will be fetched from Boot device into the internal FIFO for CPU to fetch and execute. The DRAM timing and boot device timing configuration to load BLD (Boot Loader) firmware into DRAM aredone in the BST firmware execution stage. And in the next stage, BLD firmware will be in charge of loading the code of the next stage from the boot device into DRAM and then to the next stage.

2.2 Boot Flow: Single OS

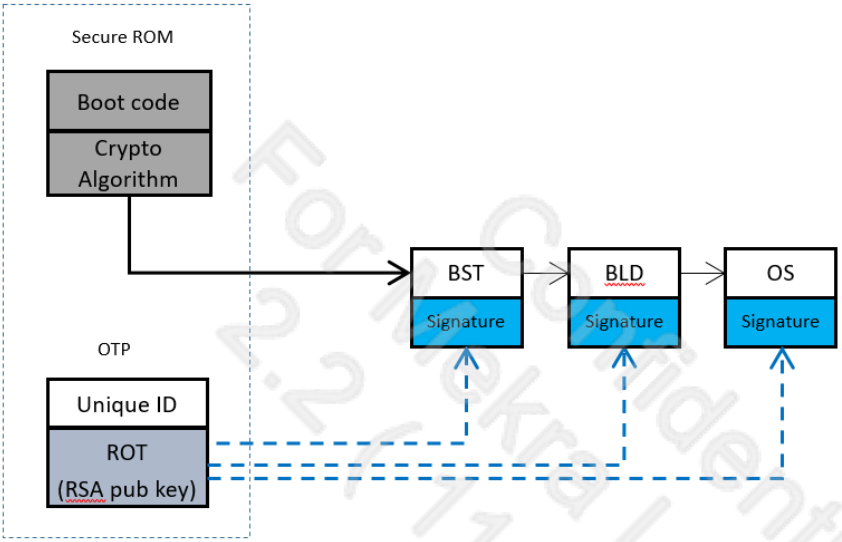


Figure 6 *Chain of Trust.*

Secure ROM is a small size ROM with pre-burned secure boot code inside. At the binary generation stage, the SHA256 (SHA512 for CV5x) is used to generate the hash of each binary/firmware, and then a private key is used to encrypt the SHA256 hash. At the secure boot stage, each boot procedure may use SHA256/512 to generate the binary of the hash for its latter procedure, and then compare it to the decrypted hash (with the public key).

Users generate one public key and one private key. For the different binaries of each boot procedure, users use its private key to generate a signature of the binary and attach this signature in the end of the binary, thus, making the binary verifiable. The public key should be programmed into the OTP ROM, so that the secure boot procedure can use it to check the integrity of its latter procedure.

Confidential
For Hekra Langley
2.2.f 110520 by

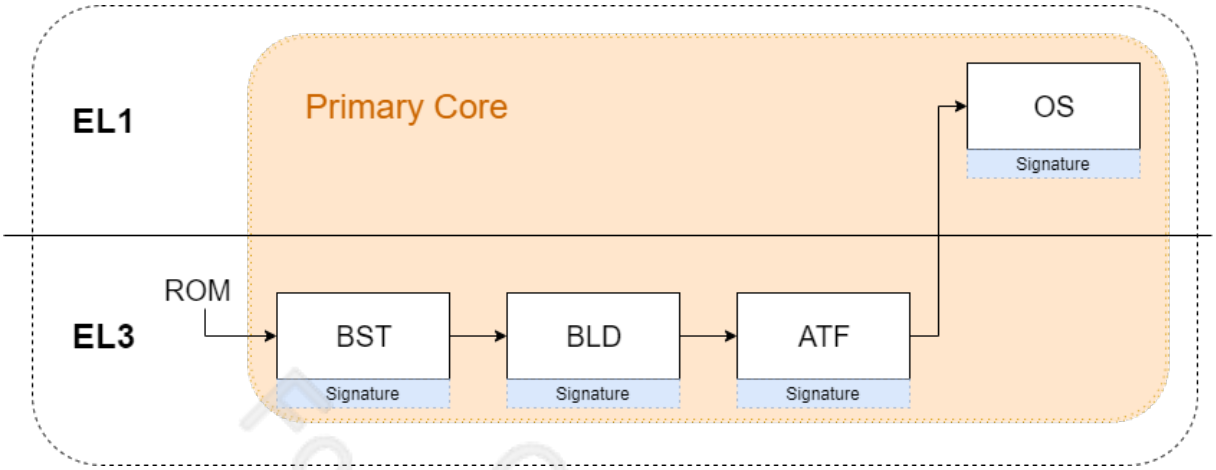


Figure 7 *Non-CV2xfs flow without OP-TEE*

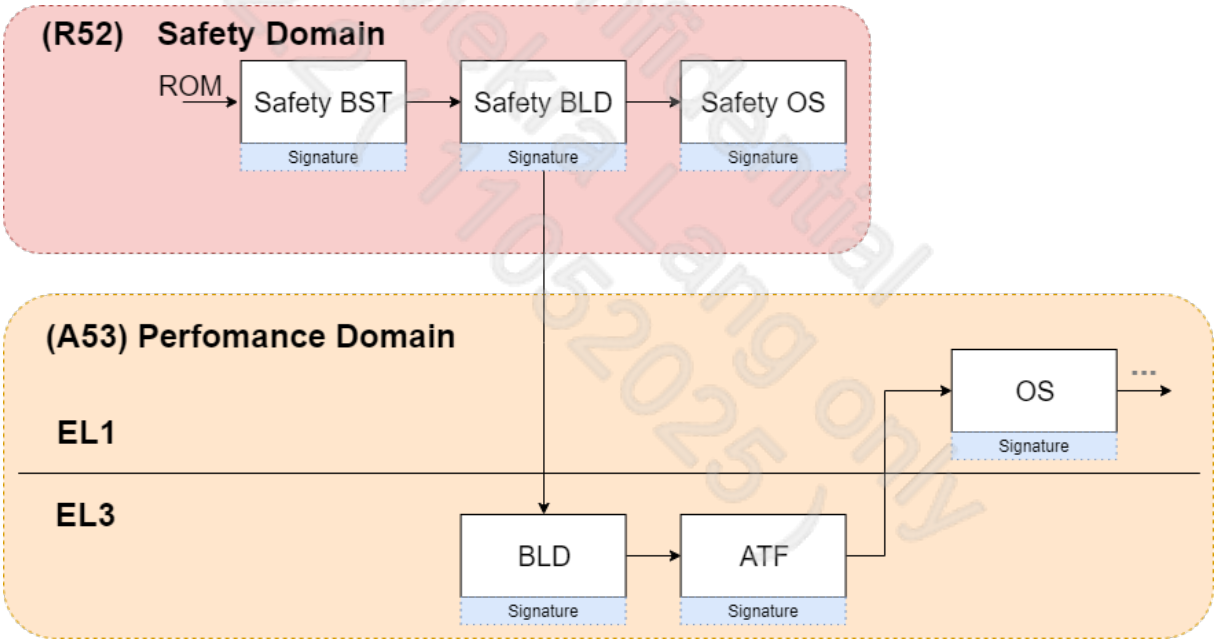


Figure 8 *CV2xfs flow without OP-TEE*

During the secure boot process, the codes in secure ROM would read the public key from the OTP and use it to check the integrity of the BST binary. In BST, there are codes to use this public key (or the derived public key from this public key) to check the integrity of the BLD firmware.

The public key must be programmed into the OTP ROM before the secure boot process, or the certification flow will be bypassed. The first running code in secure ROM also has digital signature authentication capability.

BLD will then check the integrity of next image, which could be the OS image or the QNX IPL image.

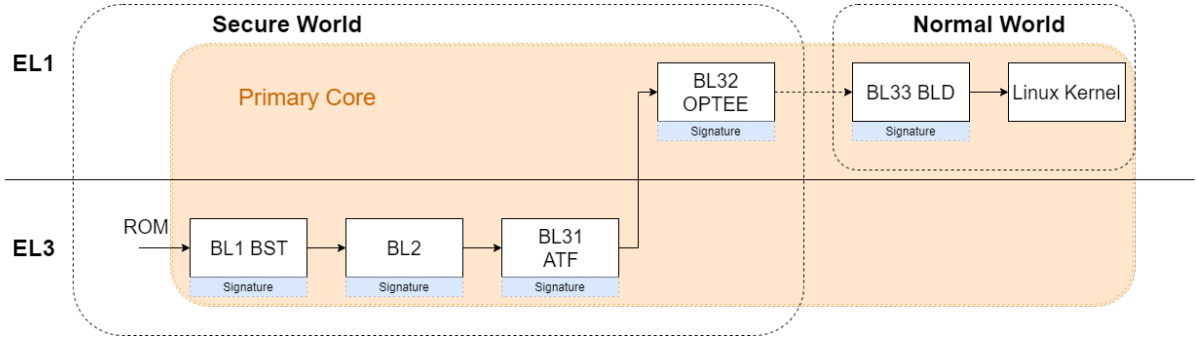


Figure 9 *Non-CV2xfs Boot flow with OP-TEE*

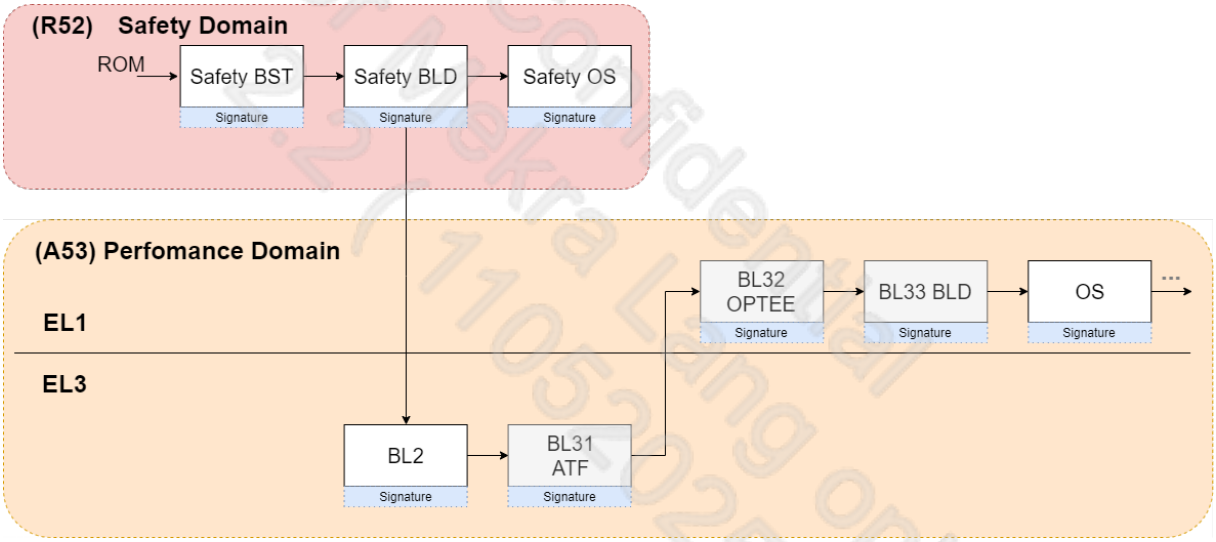


Figure 10 *CV2xfs Boot flow with OP-TEE*

For the OP-TEE case, the secure boot process is divided into several Boot stages, BL1 for BST, BL2 for the Loader of BL3 images, BL31 for Arm Trusted Firmware (open-source Secure Monitor), BL32 for OP-TEE (open-source TEE), and BL33 for Boot Loader. FIP (Firmware Image Package) is a packaging format implemented by ATF to package firmware images in a single binary. FIP normally includes BL31 Secure Monitor Firmware, BL32 OP-TEE image, and BL33 BLD image. FIP image is recognized by BL2, each image in the FIP is loaded and verified by BL2.

2.3 Boot Flow: Ambalink Dual-OS

Ambalink is the name for Ambarella AMP (Asymmetric MultiProcessing) software architecture, which enables multiple Operating Systems on the Ambarella SoCs.

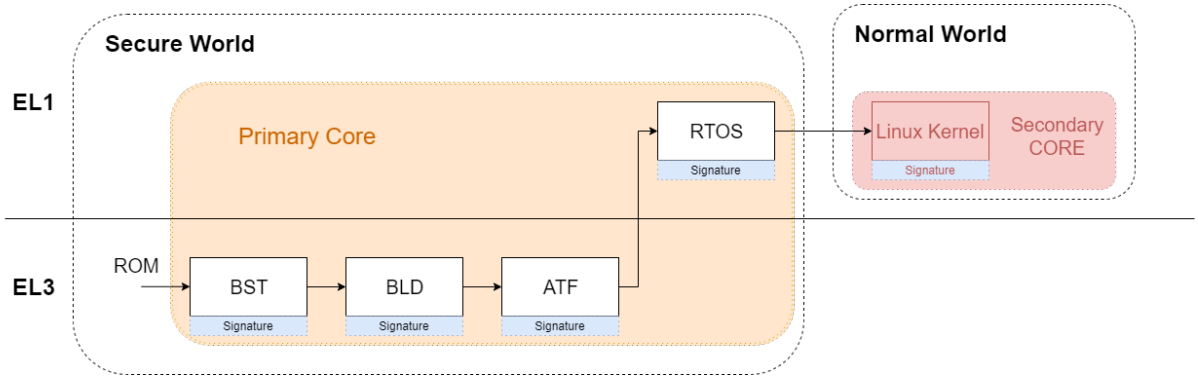


Figure 11 *Ambalink Boot Flow without OP-TEE.*

All signatures in the figure above are only needed when the secure boot is enabled. In normal boot/non-secure boot mode, the boot sequence is the same, but signature verification is not needed.

When a chip is released from the reset state, BST binary will be fetched from boot device into the internal FIFO for CPU to fetch and execute. If secure boot is enabled, the ROM code will verify the BST signature, i.e., secure ROM reads the public key from the OTP ROM and then uses the key to check the integrity of the BST binary. Then, the BST binary uses the public key (or the derived public key from this public key) to check the integrity of the next image and form the chain of trust.

For CV2fs/CV22fs chip, the BST runs in Cortex-R52, not Cortex-A53.

When the system integrator generates the firmware binary to program into the external Flash ROM of the target device, SHA256 is used to generate the hash of each binary / firmware, and then uses RSA to encrypt the hash with a private key. After the firmware binary is deployed into the Flash ROM and the device is booted with secure boot enabled, each boot stage uses the SHA256 to generate the hash of next stage of loaded binary, and then compares it with the decrypted SHA256 hash (with the public key) which is attached with the loaded binary firmware.

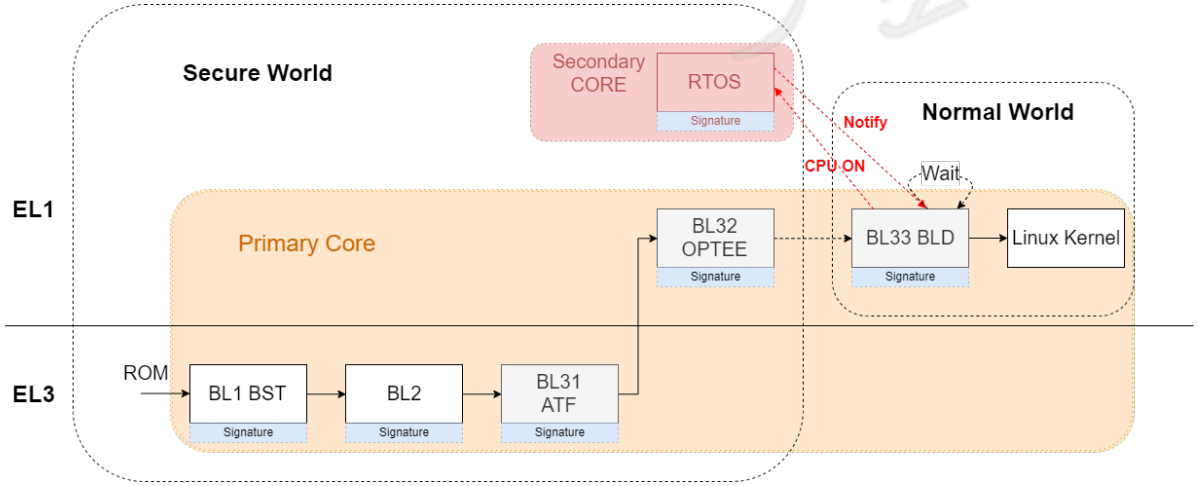


Figure 12 *Ambalink Boot Flow with OP-TEE.*

For the OP-TEE case, the secure boot process is divided into several Boot stages, BL1 for BST, BL2 for the Loader of BL3 images, BL31 for Arm Trusted Firmware (open-source Secure Monitor), BL32 for OP-TEE (open-source TEE), and BL33 for Boot Loader. FIP (Firmware Image Package) is a packaging format implemented by ATF to package firmware images in a single binary. FIP normally includes BL31 Secure Monitor Firmware, BL32 OP-TEE image, and BL33 BLD image. FIP image is recognized by BL2, each image in the FIP is loaded and verified by BL2.

For Ambalink Dual-OS, the primary core stops at BL33 Boot Loader, when the REE (Rich Execution Environment) is needed. RTOS will notify BL33 Boot Loader to boot Linux.

2.3.1 Resource Assignment

Linux is the Rich Execution Environment (REE), which runs in the normal world, and Linux running at Arm EL1 (Kernel Space) and Arm EL0 (User Space) has hardware-limited access to system or security-critical registers or devices. OP-TEE and RTOS run in the secure world, and they can have full access to the system. Primary core (core 0) runs OP-TEE and Linux. RTOS cannot run at primary core (core 0). RTOS and OP-TEE cannot run at the same CPU core because they are both secure OS and there is no EL2 (hypervisor mode) in Ambarella SDK.

2.3.2 Image Load Sequence

BL2 is loaded by BST, and FIP image (BL31, BL32, BL33) are loaded by BL2.

RTOS image is loaded by BL33. After entering OS, other system resources may also need protection. For example, if CV image and DSP image need to be protected, the images can be loaded by secure OS into the secure memory region.

2.3.3 Trust Sequence

OTP stores the root public key. The root public key authenticates the BST, and BST verifies ATF (BL2) signature. BL2 verifies all BL3x in the FIP image. The ATF constructs the chain of trust through X509v3 certificate's extension (store digest of hash, public key, non-volatile counter). In CV2x/CV2xfs/H32, all digital signature algorithm is RSA. In CV5x, BST and BL2 are verified by Ed25519 and FIP image (BL31, BL32, and BL33) and all later images are verified by RSA.

2.3.4 IPC

Two IPC mechanism used in Ambarella SDK are introduced in the following Sections.

2.3.4.1 IPC between Linux and OP-TEE

The IPC between Linux and OP-TEE needs to go through the monitor mode to switch between the normal world and the secure world. Please refer to Arm Trustzone Whitepaper for more details.

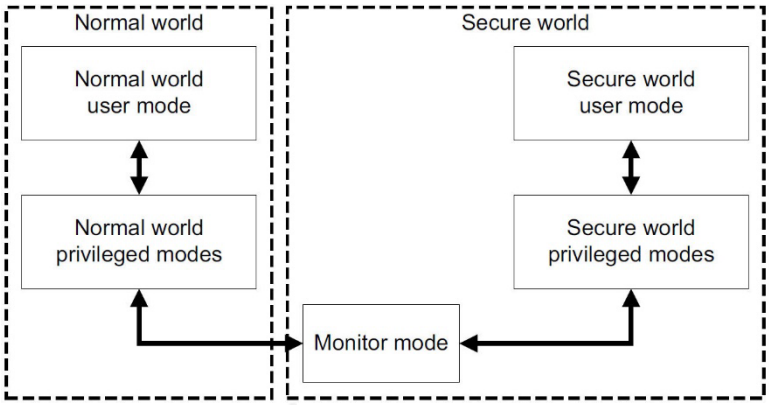


Figure 13 *IPC Flow from* Trustzone Whitepaper.

2.3.4.2 IPC between Linux and RTOS

Unlike the case of OP-TEE, Linux and RTOS run on different physical CPU core(s). This is a common approach of AMP (Asymmetric MultiProcessing) software architecture. The framework was named Ambalink since 2012, when it was first created by Ambarella for Linux to communicate with ThreadX. It is similar to the well-known OpenAMP (Open Asymmetric Multi Processing) open-source project, which was created in 2014.

The IPC method has less overhead since it does not need to go through Monitor mode.

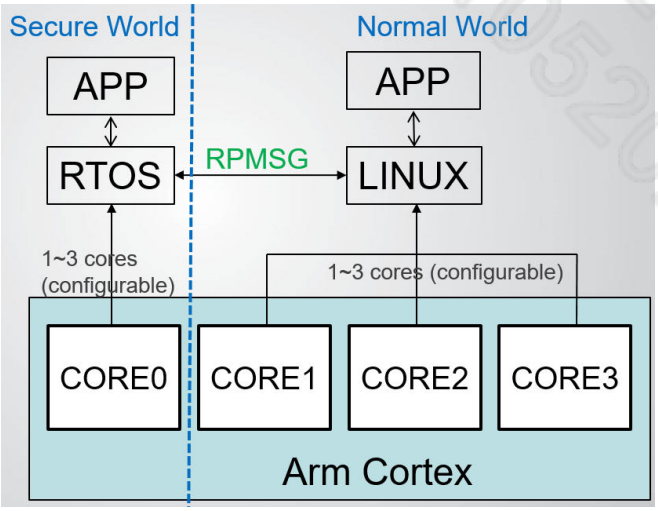


Figure 14 *Ambalink IPC.*

For Media Lang only (10/5/2025) Confidential

3 Configuration Setup

This chapter introduces SDK setup commands and the configuration options related to SDK implementation.

3.1 Generate RSA Key

Here is the demonstration for some well-known “ssh-keygen” (in openssh package) and “openssl” (in libopenssl package) commands commonly used in Linux shell prompt.

1) Generate the root PEM RSA private key:

```
$ ssh-keygen -t rsa -b 2048 -m PEM
```

- Private key is saved as id_rsa file
- Key size is 2048-bit.
- From OpenSSH 7.8 the default format RSA key pair has changed to OpenSSH format instead of PEM, need to add -m option to specify PEM format

2) Generate the root RSA public key from its private key with openssl:

```
$ openssl rsa -in id_rsa -pubout > id_rsa.pub.openssl
```

3) View private key information:

```
$ openssl rsa -noout -text -in id_rsa -modulus
```

- publicExponent: 65537 (0x10001)
- Modulus (N) value is also displayed. Note it is a 2048-bit (256-byte) large integer.

4) Generate digital signature of the file named image.bin:

```
$ openssl dgst -sha256 -sign id_rsa -out image.bin.sig
image.bin
```

- Use sha256 digest
- Specify private key filename: id_rsa
- Save signature into image.bin.sig

3.2 Generate Ed25519 Key

Here is the demonstration for some well-known “openssl” commands commonly used in Linux shell prompt.

1) Generate the ed25519 private key file named ecc_25519_private.pem in PEM format

```
$ openssl genpkey -algorithm ed25519 -outform PEM -out
ecc_25519_private.pem
```

2) Print the plain-text public key

```
$ openssl pkey -in ecc_25519_private.pem -pubout -text
-----BEGIN PUBLIC KEY-----
MCowBQYDK2VwAyEA8LRnGL657ZHA9sXyjvnOUd7qHkezvp7kcYxtqc32D+A=
```

```
-----END PUBLIC KEY-----
ED25519 Public-Key:
pub:
    f0:b4:67:18:be:b9:ed:91:c0:f6:c5:f2:8e:f9:ce:
    51:de:ea:1e:47:b3:be:9e:e4:71:8c:6d:a9:cd:f6:
    0f:e0
```

3) Generate digital signature (output file named image.bin.sig) of the file named image.bin:

```
$ openssl pkeyutl -sign -inkey ecc_25519_private.pem -out image.bin.sig -
rawin -in image.bin
```

3.3 OP-TEE

The following steps demonstrate some well-known OP-TEE test commands on Linux.

1) Install kernel driver

```
$ modprobe optee
```

2) Run tee supplicant

```
$ tee-supplicant &
```

3) Run hello world test

```
$ optee_example_hello_world
Invoking TA to increment 42
TA incremented value to 43
```

4) Run secure storage test

```
$ optee_example_secure_storage
Prepare session with the TA

Test on object "object#1"
- Create and load object in the TA secure storage
- Read back the object
- Delete the object

Test on object "object#2"
- Object not found in TA secure storage, create it.

We're done, close and release TEE resources
```

Confidential For Mekra Lang only (2025/10)

3.4 OTP

OTP is the One-Time Programmable ROM. The programmed public key acts as the Root of Trust in the secure boot flow to form the chain of trust. The content would be programmed in Ambarella or when customers manufacture by the specific purpose firmware and can only be programmed once.

Public keys is pre-burned insides OTP ROM, but the private keys should be kept safe by users and not stored on the device or leaked to others.

The OTP has a field reserved for chip specific unique ID. Users can customize the unique ID while keeping Ambarella's unique ID. The unique ID provides necessary security, and can be used for per-unit license, device authentication, anti-hardware cloning, and more.

The following table displays an example of OTP data format. Different colors indicate different region types: red means it must be programmed by customers, blue means optional field by customer, and black means reserved for customer or Ambarella. For safety reason (CRC, Parity, etc.), CV2xfs has different OTP layout from CV2x.

Bit Address (DEC)	Bit Address (HEX)	Bits	Function
0 ~ 16127	0x0000 ~ 0x3EFF	16128	Reserved by Ambarella (include 128 bit Amba unique ID)
16128 ~ 16255	0x3F00 ~ 0x3F7F	128	Customer ID (HW serial number)
16256 ~ 16383	0x3F80 ~ 0x3FFF	128	Reserved
16384 ~ 16639	0x4000 ~ 0x40FF	256	HUK for OPTEE
16640 ~ 16895	0x4100 ~ 0x41FF	256	Monotonic Counter
16896 ~ 17919	0x4200 ~ 0x45FF	256 x 4	AES keys (customized apps use)
17920 ~ 18943	0x4600 ~ 0x49FF	256 x 4	ECC keys (customized apps use)
18944 ~ 20479	0x4A00 ~ 0x4FFF	256 x 6	Reserved
20480 ~ 32767	0x5000 ~ 0x7FFF	4096x3	2048bit RSA key (Secure Boot ROT, Mandatory)

Table 2 cv2x OTP Format.

Bit Address (DEC)	Bit Address (HEX)	Bits	Function
0 ~ 16127	0x0000 ~ 0x3EFF	16128	Reserved by Ambarella (include 128 bit Amba unique ID)
32256~ 32383	0x7E00 ~ 0x7E7F	128	Customer ID (HW serial number)
32512 ~ 32639	0x7F00 ~ 0x7F7F	128	Reserved
32768 ~ 33023	0x8000 ~ 0x80FF	256	HUK for OPTEE
33280 ~ 33535	0x8200 ~ 0x82FF	256	Monotonic Counter
33792~ 34815	0x8400 ~ 0x87FF	256 x 4	AES keys (customized apps use)
35840~ 36863	0x8C00 ~ 0x8FFF	256 x 4	ECC keys (customized apps use)

37888 ~ 39423	0x9400 ~ 0x99FF	256 x 6	Reserved
40960 ~ 65535	0xA000 ~ 0xFFFF	4096x3	2048bit RSA key (Secure Boot ROT, Mandatory)

Table 3 cv2xfs OTP Format.

Bit Address (DEC)	Bit Address (HEX)	Bits	Function
0-255	0x0000-0x00FF	256	Reserved by Ambarella 0x6 Secure boot 0x3f JTAG disable 0xc0-0xdf lock bits area (0xc8 for ROTPK lock)
256-383	0x100-0x17F	128	Amba Unique ID
384-511	0x180-0x1FF	128	Reserved for user planted Seed & CUK
512-48767	0x200-0xBE7F	48256	Reserved by Ambarella 0x3e91 disable secure usb boot
48768-48799	0xBE80-0xBE9F	32	Key revoke bits
48800-48895	0xBEA0-0xBEFF	96	Reserved for BST version count
48896-49023	0xBF00-0xBF7F	128	Customer ID
49024-49151	0xBF80-0xBFFF	128	Reserved for OTP TEST
49152-49407	0xC000-0xC0FF	256	HUK for OPTEE
49408-49663	0xC100-0xC1FF	256	Reserved for user planted Seed
49664-51711	0xC200-0xC5FF	256x4	AES keys
50688-50943	0xC600-0xC9FF	256x4	ECC keys
51712-53247	0xCA00-0xCFFF	256x6	Reserved for USER
53248-57343	0xD000-0xDFFF	256x16	Ed25519 Root of Trust Public keys
57344-57599	0xE000-0xE0FF	256	MONO counter0
57600-57855	0xE100-0xE1FF	256	MONO counter1
57856-58367	0xE200-0xE3FF	512	MONO counter2
58368-64511	0xE400-0xFBFF	6144	Reserved for USER

Table 4 CV5x OTP Format.

The OTP APIs are as follows.

In OPTEE OS:

```
/* Write Public key into OTP */
#define AMBA_OTP_CMD_WRITE_PUBKEY   0
/* Read Public key from OTP */
```

```
#define AMBA_OTP_CMD_READ_PUBKEY    31
/* Read AMBA Unique ID in OTP */
#define AMBA_OTP_CMD_READ_AMBA_UNIQUE_ID  2
/* Write Customer Unique ID into OTP */
#define AMBA_OTP_CMD_WRITE_CUSTOMER_ID  4
/* Read Customer Unique ID from OTP */
#define AMBA_OTP_CMD_READ_CUSTOMER_ID  5
/* Read the monotonic counter from OTP */
#define AMBA_OTP_CMD_READ_MONOTINIC_COUNTER  6
/* Increase the monotonic counter in OTP */
#define AMBA_OTP_CMD_INCREASE_MONOTINIC_COUNTER  7
/* Permanently Enable Secure Boot */
#define AMBA_OTP_CMD_PERMANENTLY_ENABLE_SECURE_BOOT  8
/* Generate HW Unique Encryption Key */
#define AMBA_OTP_CMD_GEN_HW_UNIQUE_ENCRYPTION_KEY  9
/* Generate AES Key */
#define AMBA_OTP_CMD_GEN_AES_KEY  10
/* Write AES Key */
#define AMBA_OTP_CMD_WRITE_AES_KEY  11
/* Revoke Public Key in OTP */
#define AMBA_OTP_CMD_REVOKE_KEY  18
/* Query OTP Setting */
#define AMBA_OTP_CMD_QUERY_OTP_SETTING  20
```

Check source code to see all optee commands.

For source code, please read:

optee_test/host/test_program_otp/test_program_otp.c

optee_os/core/pta/pta_ambarella_otp.c

arm-trusted-firmware/plat/ambarella/driver/ambarella_otp.c

The Linux test commands for read/increase monotonic counter are as follows.

```
/* read monotonic counter 0 in OTP */
$ test_program_otp --readmonocounter 0
/* increase monotonic counter 0 in OTP */
$ test_program_otp --increasemonocounter 0
```

The Linux test commands for OTP programming are as follows.

```
/* write pubkey.bin to key0 in OTP, no lock */
$ test_program_otp --writepubkey 0 /tmp/SD0/pubkey.bin 0
```

Ambarella Confidential For Mekra Lang only 2.2 (1105205)

```
/* read public key0 in OTP */
$ test_program_otp --readpubkey 0
/* revoke public key1 in OTP */
$ test_program_otp --revokekey 1
/* read AMBA Unique ID */
$ test_program_otp --readambauniqueid
/* write customerid.bin to OTP *
$ test_program_otp --writecustomeruid customerid.bin
/* Permanent enable secure boot */
$ test_program_otp –-permanentlyenablesecureboot
/* Generate HUK */
$ test_program_otp --generatehuk
```

Check “test_program_otp --help” to see all commands.

For ThreadX to access OTP, please re-use the API in ATF. Customer needs to call the corresponding EL3 ATF SiP (Silicon Provider) SMC call. Some example code is shown as below.

Write RSA public key into OTP

```
/* int ambarella_otp_write_rot_pubkey(uint8_t *p, uint32_t length, uint32_t key_index); */
#define RTOS_AMBA_SIP_OTP_SET_PUKEY 0x82000504
static volatile UINT8 pukey[512] __attribute__((aligned(4096)));
UINT32 key_index = 0;
AmbaSmcCall(RTOS_AMBA_SIP_OTP_SET_PUKEY, pukey, sizeof(pukey), key_index, 0);
```

Read RSA public key from OTP

```
/* int ambarella_otp_read_rot_pubkey(uint8_t *p, uint32_t length, uint32_t key_index); */
#define RTOS_AMBA_SIP_OTP_GET_PUKEY 0x82000503
static volatile UINT8 pukey[512] __attribute__((aligned(4096)));
UINT32 key_index = 0;
AmbaSmcCall(RTOS_AMBA_SIP_OTP_GET_PUKEY, pukey, sizeof(pukey), key_index, 0);
```

Lock RSA public key0

```
/*int ambarella_otp_lock_rot_pubkey(uint32_t key_index, uint32_t simulate);*/
#define RTOS_AMBA_SIP_OTP_LOCK_PUKEY 0x82000533
UINT32 key_index = 0;
AmbaSmcCall(RTOS_AMBA_SIP_OTP_LOCK_PUKEY, key_index, 0, 0, 0);
```

Write Ed25519 public key into OTP

```
/* int ambarella_otp_write_rot_pubkey(uint8_t *p, uint32_t length, uint32_t key_index); */
#define RTOS_AMBA_SIP_OTP_SET_PUKEY 0x82000504
static volatile UINT8 pukey[32] __attribute__((aligned(4096)));
UINT32 key_index = 0;
```

```
AmbaSmcCall(RTOS_AMBA_SIP_OTP_SET_PUKEY, pukey, sizeof(pukey), key_index, 0);
```

Read Ed25519 public key from OTP

```
/* int ambarella_otp_read_rot_pubkey(uint8_t *p, uint32_t length, uint32_t key_index); */
#define RTOS_AMBA_SIP_OTP_GET_PUKEY 0x82000503
static volatile UINT8 pukey[32] __attribute__((aligned(4096)));
UINT32 key_index = 0;
AmbaSmcCall(RTOS_AMBA_SIP_OTP_GET_PUKEY, pukey, sizeof(pukey), key_index, 0);
```

Lock all Ed25519 public keys

```
/*int ambarella_otp_lock_rot_pubkey(uint32_t key_index, uint32_t simulate);*/
#define RTOS_AMBA_SIP_OTP_LOCK_PUKEY 0x82000533
UINT32 key_index = 16;
AmbaSmcCall(RTOS_AMBA_SIP_OTP_LOCK_PUKEY, key_index, 0, 0, 0);
```

Revoke public key from OTP

```
/* int ambarella_otp_revoke_key(uint32_t index, uint32_t simulate); */
#define RTOS_AMBA_SIP_OTP_REVOKE_PUKEY 0x8200052b
UINT32 key_index = 0;
AmbaSmcCall(RTOS_AMBA_SIP_OTP_REVOKE_PUKEY, key_index, 0, 0, 0);
```

Read AMBA Unique ID in OTP

```
/* int ambarella_otp_read_amba_unique_id(uint8_t *p, uint32_t length); */
#define RTOS_AMBA_SIP_OTP_GET_UNIQUE_ID 0x82000501
static volatile UINT8 amba_id[16] __attribute__((aligned(64)));
AmbaSmcCall(RTOS_AMBA_SIP_OTP_GET_UNIQUE_ID, amba_id, sizeof(amba_id), 0, 0);
```

Read the monotonic counter0 from OTP

```
/* int ambarella_otp_read_mono_counter(uint32_t *p,  uint32_t length, uint32_t mono_cnt_index); */
#define RTOS_AMBA_SIP_OTP_GET_COUNTER 0x82000507
static volatile UINT32 counter __attribute__((aligned(64)));
AmbaSmcCall(RTOS_AMBA_SIP_OTP_GET_COUNTER, &counter, sizeof(counter), 0, 0);
```

Increase the monotonic counter in OTP

```
/* int ambarella_otp_increase_mono_counter(uint32_t mono_cnt_index, uint32_t simulate); */
#define RTOS_AMBA_SIP_OTP_ADD_COUNTER 0x82000508
AmbaSmcCall(RTOS_AMBA_SIP_OTP_ADD_COUNTER, 0, 0, 0, 0);
```

Permanently Enable Secure Boot

```
/* int ambarella_otp_permanently_enable_secure_boot(uint32_t simulate); */
#define RTOS_AMBA_SIP_OTP_PERMANENTLY_ENABLE_SECURE_BOOT 0x82000509
AmbaSmcCall(RTOS_AMBA_SIP_OTP_PERMANENTLY_ENABLE_SECURE_BOOT, 0, 0, 0, 0);
```

Generate HW Unique Encryption Key

```
/*int ambarella_otp_write_huk_nonce(uint8_t *p, uint32_t length);*/
#define RTOS_AMBA_SIP_OTP_SET_HUK_NONCE 0x82000521
```

For Mekra Lang only (2021-1105-2023) Confidential

```
static volatile UINT32 random_data[8] __attribute__((aligned(64)));
AmbaSmcCall(RTOS_AMBA_SIP_OTP_SET_HUK_NONCE, random_data, sizeof(random_data), 0, 0);
```

Read HW Unique Encryption Key

```
/*int ambarella_otp_read_huk(uint8_t *p, uint32_t length);*/
#define RTOS_AMBA_SIP_OTP_GET_HUK 0x82000520
static volatile UINT8 amba_huk[16] __attribute__((aligned(64)));
AmbaSmcCall(RTOS_AMBA_SIP_OTP_GET_HUK, amba_huk, sizeof(amba_huk), 0, 0);
```

Check “arm-trusted-firmware/plat/ambarella/include/ambarella_smc.h” for all SMC call defined numbers.
Check “arm-trusted-firmware/plat/ambarella/include/plat_private.h” for the SMC call arguments.

Confidential For Miekra Lang only (2025-7-10)

For QNX to access OTP, please refer to AmbaOTP.h

```
/* Read public key from OTP */
UINT32 AmbaOTP_PublicKeyGet(UINT8 *Buffer, UINT32 BufferSize, UINT32 KeyIndex, UINT32 *KeyStatus);
/* Write Public key into OTP */
UINT32 AmbaOTP_PublicKeySet(UINT8 *Buffer, UINT32 BufferSize, UINT32 KeyIndex);
/* Revoke public key from OTP */
UINT32 AmbaOTP_PublicKeyRevoke(UINT32 KeyIndex);
/* Read AMBA Unique ID in OTP */
UINT32 AmbaOTP_AmbaUniqueIDGet(UINT8 *Buffer, UINT32 BufferSize);
/* Write customer Unique ID into OTP */
UINT32 AmbaOTP_CustomerUniIDSet(const UINT8 *Buffer, UINT32 BufferSize);
/* Read Customer Unique ID from OTP */
UINT32 AmbaOTP_CustomerUniIDGet(UINT8 *Buffer, UINT32 BufferSize);
/* Read the monotonic counter from OTP */
UINT32 AmbaOTP_MonoCounterGet(UINT32 *Counter);
/* Increase the monotonic counter in OTP */
UINT32 AmbaOTP_MonoCounterIncrease(void);
/* Permanently Enable Secure Boot */
UINT32 AmbaOTP_SecureBootEnable(void);
/* Write AES Key */
UINT32 AmbaOTP_AesKeySet(const UINT8 *Buffer, UINT32 BufferSize, UINT32 KeyIndex);
/* Read Reserved DX */
UINT32 AmbaOTP_ReservedDXGet(UINT8 *Buffer, UINT32 BufferSize, UINT32 Index);
/* Write Reserved DX */
UINT32 AmbaOTP_ReservedDXSet(const UINT8 *Buffer, UINT32 BufferSize, UINT32 Index);
/* Read Reserved B2 */
UINT32 AmbaOTP_ReservedB2Get(UINT8 *Buffer, UINT32 BufferSize);
/* Write Reserved B2 */
```

UINT32 AmbaOTP_ReservedB2Set(const UINT8 *Buffer, UINT32 BufferSize);

3.4.1 OTP Caution

The OTP memory is One-Time Programmable memory, so its electrical characteristics will be changed after programming and cannot be erased or reprogrammed. Thus, it is highly recommended to program OTP carefully.

OTP important note for CV2x/CV2xfs/H32

- There are three public keys, and the **last** key cannot be revoked.
- Each key can be locked separately.
- If the first ROTPK is revoked, the second ROTPK is used.
- If the second ROTPK is revoked, the third ROTPK is used.

OTP important note for CV5x

- There are 16 public keys; the **first** key cannot be revoked.
- All 16 keys are locked all at once and cannot be locked separately.
- In secure-boot-enabled firmware, BL1 (BST) and BL2 (boot loader) images add extra 16 bytes of header, and the header specifies the ROTPK index to verify image signature.
- In secure-boot-enabled firmware, POC[6] MUST be set to 1, to recognize the extra 16 bytes of header
  - If the ROTPK is not locked, it will be normal boot, even in secure-boot-enabled firmware.
  - If the ROTPK is locked, and the image signature verification is OK, it will boot.
  - If the ROTPK is locked, and the image signature verification is not OK, it will not boot

4 iCam SVC Secure Boot

4.1 How to Generate and Burn OTP Public Key (CV5x)

1) Generate the Ed25519 private key file named ecc_25519_private.pem in PEM format

```
$ openssl genpkey -algorithm ed25519 -outform PEM -out
ecc_25519_private.pem
```

2) Print the plain-text public key

```
$ openssl pkey -in ecc_25519_private.pem -pubout -text
-----BEGIN PUBLIC KEY-----
MCowBQYDK2VwAyEA3/Eh4+AFmDZhtgjxTOuC4AyH+Q6cycJXoZ+1KFF9bMk=
-----END PUBLIC KEY-----
ED25519 Public-Key:
pub:
    df:f1:21:e3:e0:05:98:36:61:b6:08:f1:4c:eb:82:
    e0:0c:87:f9:0e:9c:c9:c2:57:a1:9f:b5:28:51:7d:
    6c:c9
```

3) Create the pubkey.bin file. The example command to create pubkey.bin step 2 is:

```
$ echo -n -e '\xdf' >  pubkey.bin
$ echo -n -e '\xf1' >> pubkey.bin
$ echo -n -e '\x21' >> pubkey.bin
$ echo -n -e '\xe3' >> pubkey.bin
$ echo -n -e '\xe0' >> pubkey.bin
$ echo -n -e '\x05' >> pubkey.bin
$ echo -n -e '\x98' >> pubkey.bin
$ echo -n -e '\x36' >> pubkey.bin
$ echo -n -e '\x61' >> pubkey.bin
$ echo -n -e '\xb6' >> pubkey.bin
$ echo -n -e '\x08' >> pubkey.bin
$ echo -n -e '\xf1' >> pubkey.bin
$ echo -n -e '\x4c' >> pubkey.bin
$ echo -n -e '\xeb' >> pubkey.bin
$ echo -n -e '\x82' >> pubkey.bin
$ echo -n -e '\xe0' >> pubkey.bin
$ echo -n -e '\x0c' >> pubkey.bin
$ echo -n -e '\x87' >> pubkey.bin
$ echo -n -e '\xf9' >> pubkey.bin
$ echo -n -e '\x0e' >> pubkey.bin
$ echo -n -e '\x9c' >> pubkey.bin
$ echo -n -e '\xc9' >> pubkey.bin
$ echo -n -e '\xc2' >> pubkey.bin
$ echo -n -e '\x57' >> pubkey.bin
$ echo -n -e '\xa1' >> pubkey.bin
$ echo -n -e '\x9f' >> pubkey.bin
$ echo -n -e '\xb5' >> pubkey.bin
$ echo -n -e '\x28' >> pubkey.bin
$ echo -n -e '\x51' >> pubkey.bin
$ echo -n -e '\x7d' >> pubkey.bin
$ echo -n -e '\x6c' >> pubkey.bin
$ echo -n -e '\xc9' >> pubkey.bin
```

4) Burn OTP public key name pubkey.bin in SD card.

In ThreadX (target board), this will burn pubkey.bin into OTP public key index 0.

```
a:\> atf writepubkey c:\pubkey.bin 0
```

Confidential
For ChipLang only
(2025-07-05)

In Linux (target board), start optee (skip if optee already started):

```
$ modprobe optee
$ tee-supplicant &
```

Burn pubkey.bin into OTP public key index 0.

```
$ test_program_otp --writepubkey 0 /tmp/SD0/pubkey.bin 0
```

5) Repeat the steps to burn all the 16 public keys. In CV5x, all 16 keys share a single lock bit, and that is why users need to burn all the 16 keys before locking it.

6) Lock all public keys.

In ThreadX (target board), this will burn pubkey.bin into OTP public key index 0.

```
a:\> atf lockeccpubkey
```

In Linux (target board), this will burn pubkey.bin into OTP public key index 0.

```
$ test_program_otp -lockallrotpubkeys
```

7) Done! ROTPK has been programed into OTP.

One can start to generate firmware for secure boot.

Please remember to use external POC[6] to verify the firmware before the secure boot is permanently enabled.

4.2 How to Generate and Burn OTP Public Key (non-CV5)

1) Generate the root RSA private key (id_rsa):

```
$ ssh-keygen -t rsa -b 2048 -m PEM
```

2) Save value of Modulus into N.txt (This is the first 256-bytes of public key on the OTP ROM):

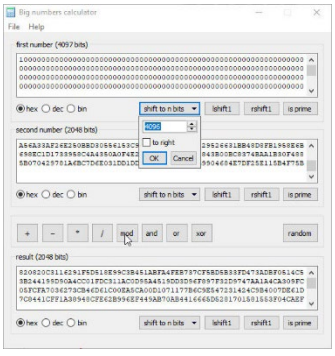
```
$ openssl rsa -noout -text -in id_rsa -modulus|grep Modulus|cut
-d '=' -f 2
```

3) Manually calculate X and generate X.txt (This is the last 256-bytes of the public key on the OTP ROM):

$$X = R^2 \bmod N,\ R = 2^{2048}$$

N is the value of Modulus.

The following example uses bncalc tool to calculate X: https://sourceforge.net/projects/bncalc/.



4) Put N.txt and X.txt in tools/hostutils/genpubkey/ and use this command to generate pubkey.bin.

```
$ ./genpubkey N.txt X.txt
```

In ThreadX (target board):

This will burn pubkey.bin into OTP public key index 0.

```
a:\> atf writepubkey c:\pubkey.bin 0
```

In QNX(target board):

This will mount the SD card to /mnt.

```
$ mount -t dos -o exe=all /dev/hd0t12 /mnt/
```

This will burn pubkey.bin into OTP public key index 0.

```
$ diag-ambarella otp write_key C1 /mnt/pubkey.bin
```

5) Done! ROTPK has been programed into OTP.

One can start to generate firmware for secure boot by configuring one's private key path in SDK configure option CONFIG_RSA_PRIVATE_KEY_PATH.

Please remember to use external POC[6] to verify one's firmware before one permanently enables secure boot.

4.3 Compile Secure Boot SVC

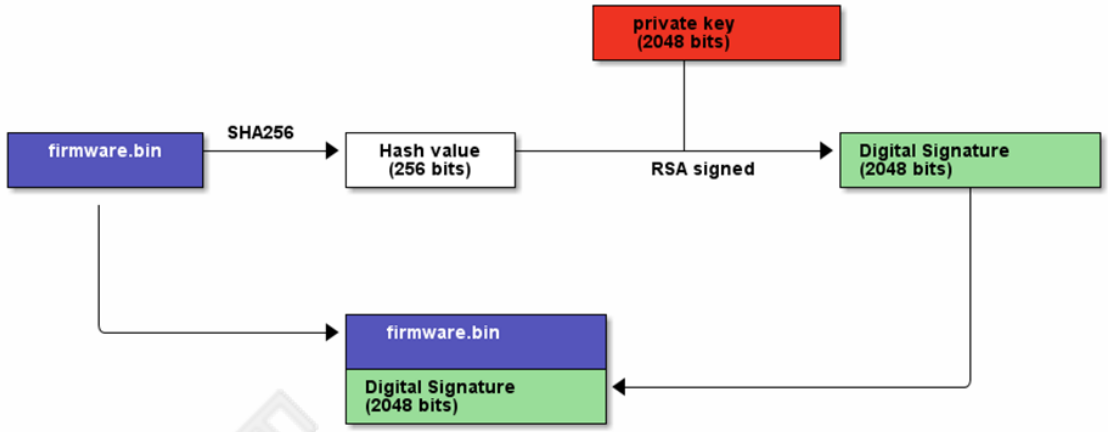
Follow the steps below to build the secure boot FW.

4.3.1 CV22

```
build$ source ./build/maintenance/env/env_set.sh cv22
build$ make cv22/icam/tx32/cv22_tx32_dk_icam_cv_nand_lt6911_secure_boot_defconfig
build$ make -j
```

The compiled binary is `bst_bld_atf_sys_rom_dsp.elf` in the output image folder. The firmware file build flow with signature is shown as follows.

For Mekra Lang only
Confidential
2025/10/15



The configuration makes BOTH Secure Boot Option and mbedtls FOSS enabled. The “RSA_PRIVATE_KEY” field is assigned to the private key (generated in Section 3.1) location.

```
build$ make menuconfig
-*- Boot Configuration --->
-*- Secure Boot Option --->
  *** This page only works with secure booting mode. ***
  *** Don't enable it if your board configuration is normal booting. ***
  [*] Enable system security service
  (${srctree}/svc/apps/icam/key/id_rsa) RSA private key file path
  () LNX source path
```

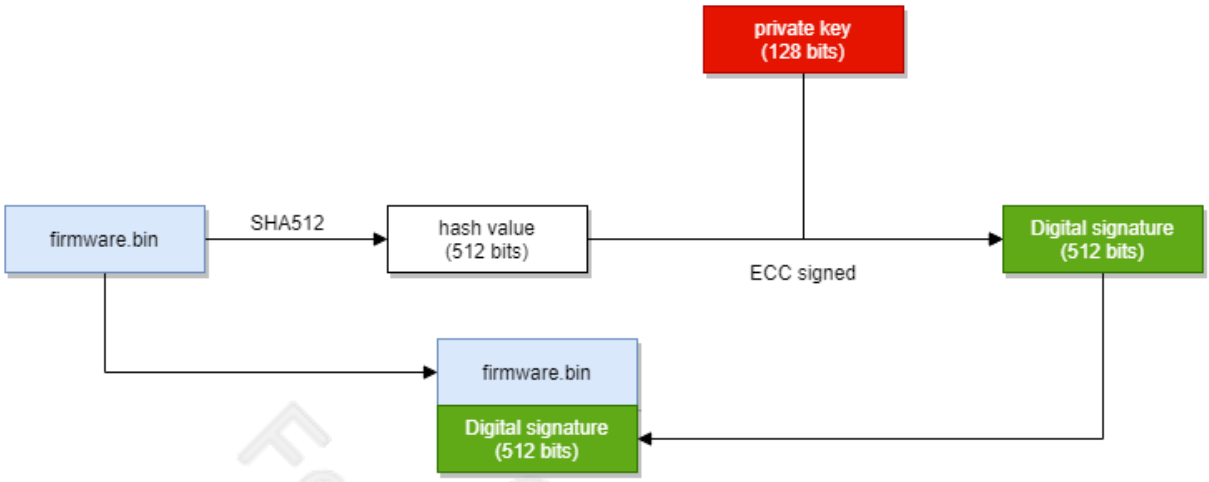
```
build$ make menuconfig
-*- FOSS Options  --->
[*] Build mbedtls FOSS library
```

4.3.2 CV5

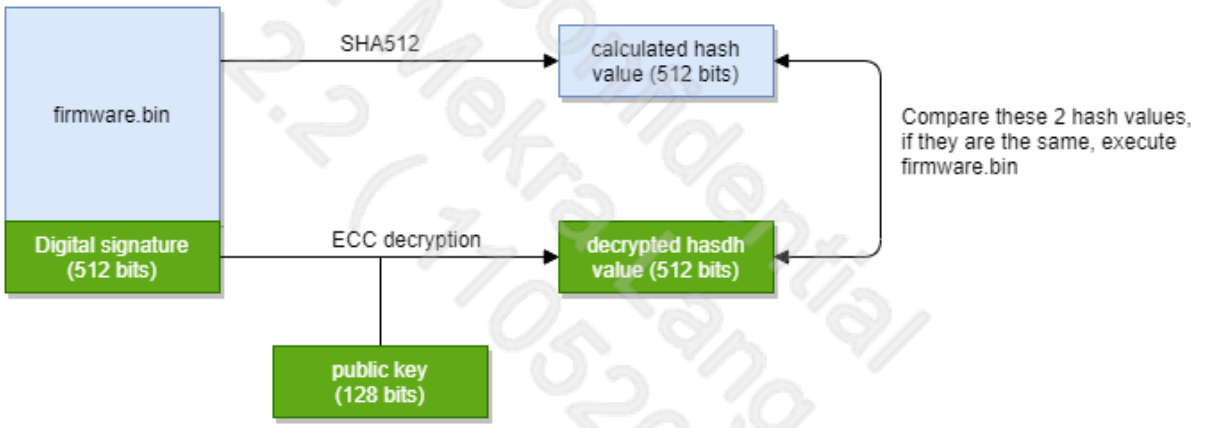
```
build$ source /opt/cvtools/env/env_set.sh cv5
build$ alias tmake='make CROSS_COMPILE=aarch64-none-elf-'
build$ tmake cv5/icam/tx64/ cv5_tx64_dk_a_icam_cv_spinand_imx577_secure_boot_defconfig
build$ tmake diags; tmake -j
```

The compiled binary is `bst_bld_atf_sys_xtb_dsp_rom.elf` in the output image folder. The firmware file build flow with signature is shown as follows.

For Mekra Lang only (2025-05-2) Confidential



The firmware load and verify flow is shown as below figure.



The configuration makes Secure Boot Option, Boot up ATF, Boot Secure RTOS after ATF, ARM trusted firmware, mbedtls FOSS and Build ATF from source enabled. The “RSA_PRIVATE_KEY” field is assigned to the private key (generated in Section 3.1) location and the “ECC_PRIVATE_KEY” field is assigned to the private key (generated in Section 3.2) location.

```
build$ make menuconfig
-*- Boot Configuration --->
-*- Secure Boot Option --->
  *** This page only works with secure booting mode. ***
  *** Don't enable it if your board configuration is normal booting. ***
  [*] Enable system security service
  (${srctree}/svc/apps/icam/main/key/id_rsa) RSA privare key file path
  (${srctree}/svc/apps/icam/main/key/ecc_25519_private.pem) ECC privare key file path
  (/usr/local/bin/openssl) openssl path
  () LNX source path
  (0)   Index/Id of key used
```

```
build$ make menuconfig
-*- Boot sequence after amboot (Boot up ATF by amboot)  --->
```

(X) Boot up ATF by amboot

```
build$ make menuconfig
-*- Boot sequence after ATF at A53 (Boot Secure RTOS atfer ATF)  --->
(X) Boot Secure RTOS atfer ATF
```

```
build$ make menuconfig
-*- Firmware Partition Manager  --->
[*] Enable ARM Trusted Firmware
```

```
build$ make menuconfig
-*- FOSS Options  --->
[*] Build mbedtls FOSS library
```

```
build$ make menuconfig
-*- ARM-Trusted-Firmware  --->
[*] Build ATF from source
  [ ]   Debug information
  [ ] Have ATF BL2
  (0x00004000) Base Address of ATF(BL31)
  (0x0003C000) Size of ATF(BL31)
  [ ] Have OP-TEE
  [ ] Support suspend to SRAM and enter DDR self-refresh
```

Ambarella Confidential
For Mekra Lang only
2022 (1105025)

4.4 Implementation with mbedtls FOSS Library

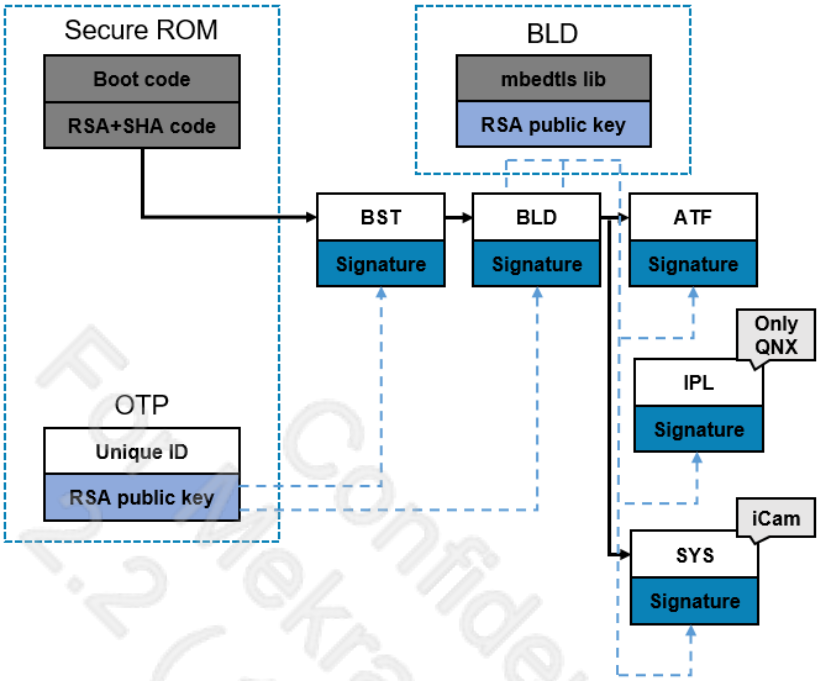


Figure 15 *CV2x Verification of ATF/IPL/SYS Binary.*

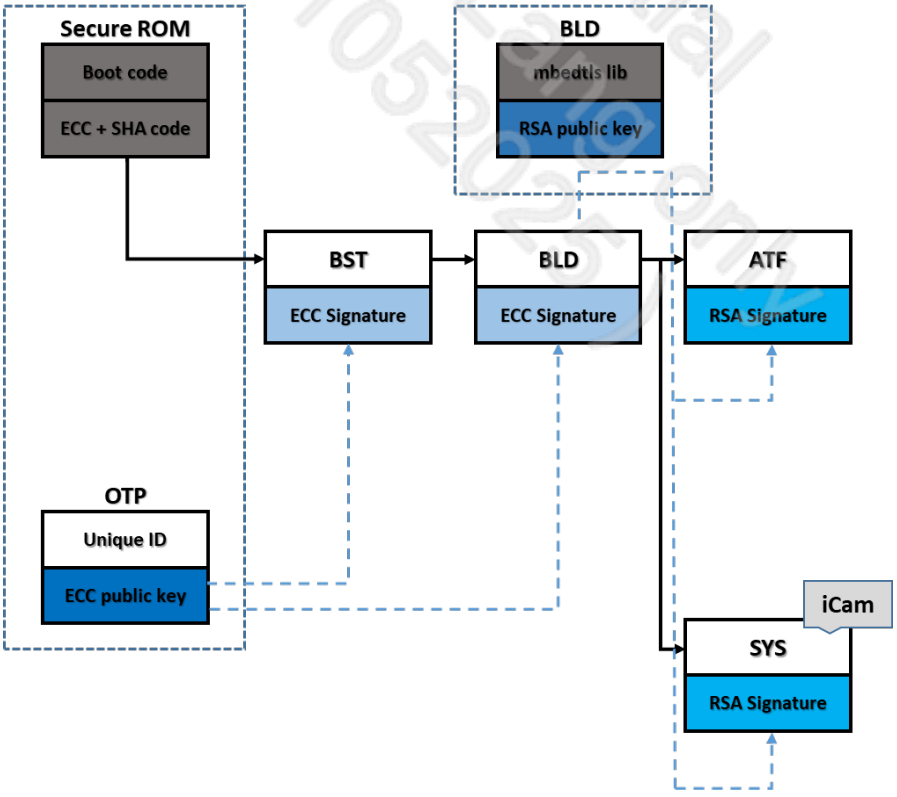


Figure 16 CV5 Verification of ATF/SYS Binary.

Different from the integrity of the BST binary and BLD binary implemented by secure ROM and OTP, the integrity checking of the ATF and SYS binary is implemented in the BLD binary. It is verified in the previous stage with the assistance of the mbedtls FOSS library.

If the secure boot is enabled, the FOSS library will be included in the BLD binary and used for the RSA digital signature of the SYS binary. Because SYS binary will be modified frequently, the public key will be stored in the BLD binary, and it is programmable (changeable) during development.

4.5 Build Secure Boot Linux Kernel (non-CV5)

By default, only Linux kernel binary will not be generated if secure boot is enabled. Optionally, Linux kernel binary can be generated with RSA digital signature appended following the steps below.

```
build$ make menuconfig
-*- Boot Configuration --->
-*- Secure Boot Option --->
  *** This page only works with secure booting mode. ***
  *** Don't enable it if your board configuration is normal booting. ***
  [*] Enable system security service
   (${srctree}/svc/apps/icam/key/id_rsa) RSA private key file path
   (${srctree}/svc/comsvc/ambalink/linux_image/cv22/icam/Image) LNX source path
```

The red highlighted part, “Linux source path”, will be the path of Linux source binary. The binary file is provided by SDK and can be replaced by the user. The file is used to generate the RSA digital signature. After compiling, another binary file, which is programmed to flash, will be generated automatically.

```
build$ make menuconfig
-*- Firmware Partition Manager  --->
-*- Linux Kernel Partition  --->
  (${srctree}/svc/comsvc/ambalink/linux_image/cv22/icam/Image_Sig.bin) Linux image file
  (1.0.0) Linux version number
  (0x5a680000) Load address
  (${srctree}/svc/comsvc/ambalink/linux_image/cv22/icam/cv22bub_ambalink.dtb) DTB image file
```

The red highlighted part “Linux image file”, will be the target output binary generated after compiling. Note that “LNX source path” for security boot cannot be the same with “Linux image file” or the compiling process will return error.

4.6 Build Secure Boot DSP uCode and FlexiBin in SECURE Partition

In the latest SDK8 SVC iCam application, the original “DSP_uCode” partition is replaced by a new partition called “SECURE” partition. During the compiling process, the files in the security folder will be automatically encrypted with the RSA algorithm to generate the signature to append to the original files. These files are going to be used to generate the SECURE partition.

Confidential
For Marketing only
Ver. 1.0 2023-07-05

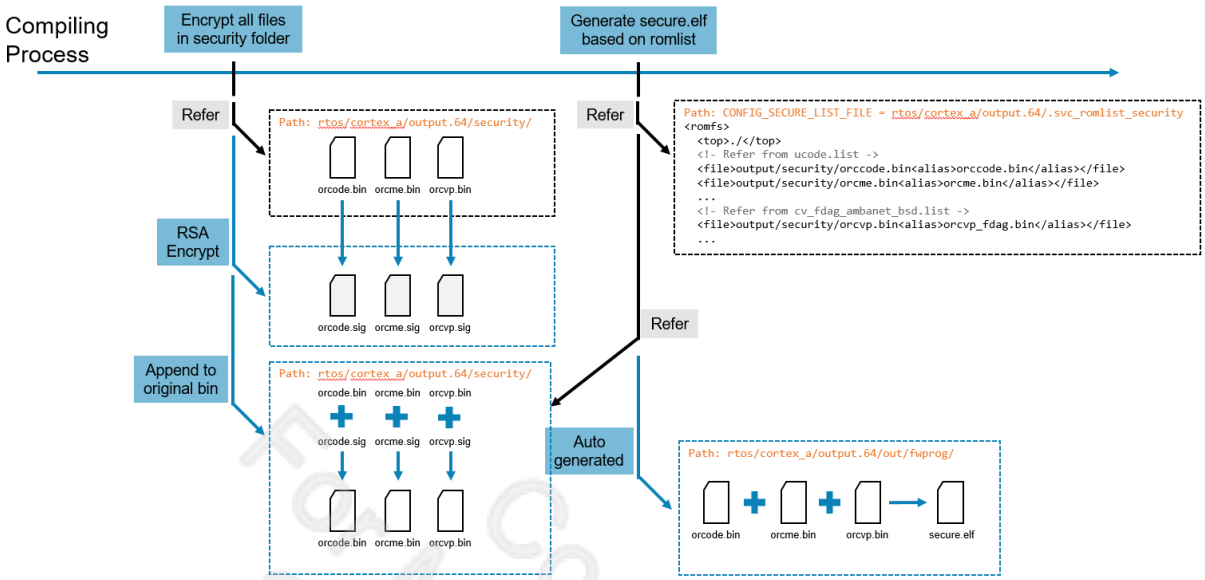


Figure 17 *Verification of ATF/IPL/SYS Binary.*

After booted, iCam will load the left binary in the SECURE partition and SYS_DATA partition from NAND into DRAM. All data in the SECURE partition will be loaded to DRAM and verified with RSA to check the integrity if secure boot is enabled. Nevertheless, the data stored in SYS_DATA partition will be directly loaded to DRAM.

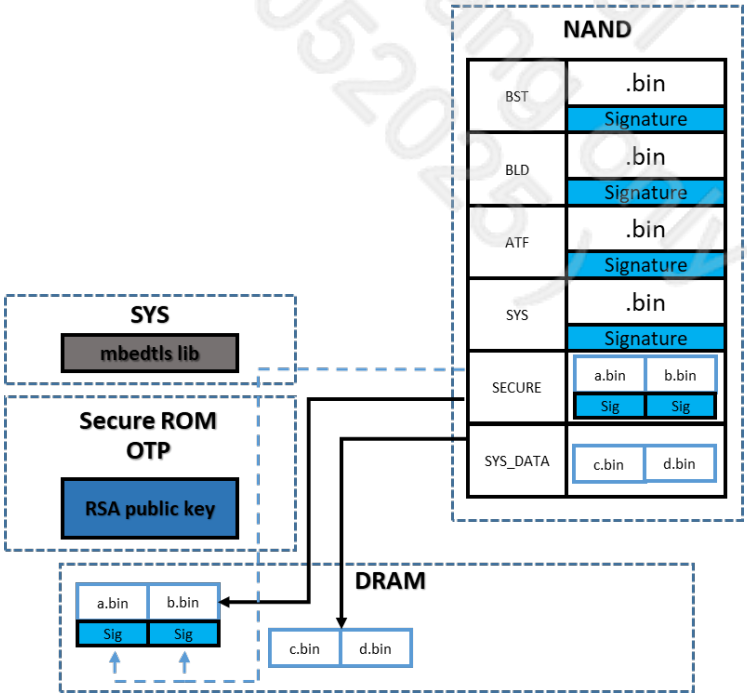


Figure 18 CV2x Verification of SECURE and SYS_DATA Binary.

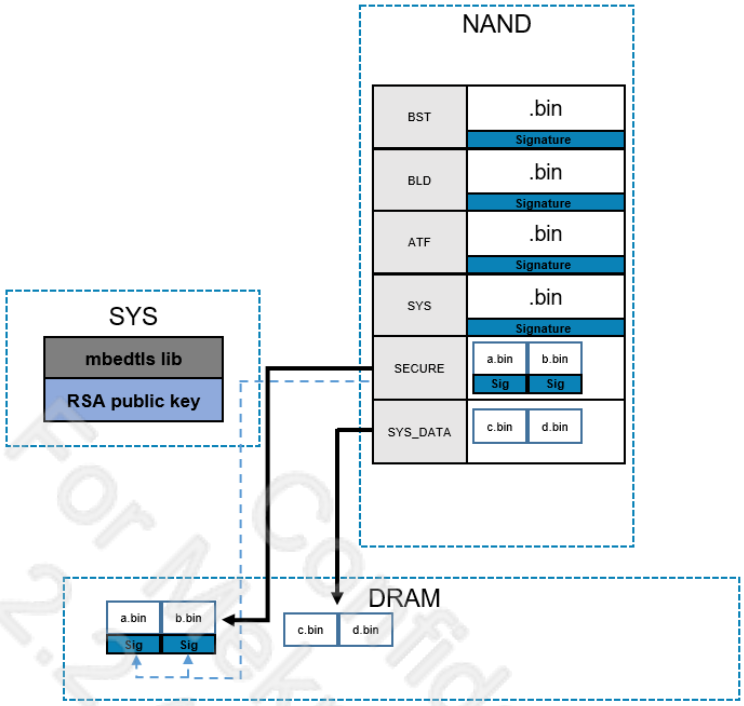


Figure 19 *CV5 Verification of SECURE and SYS_DATA Binary.*

Regarding to the RSA public key to verify SECURE partition, the CV2x SYS reads it from OTP and the CV5x SYS itself contains it.

5 Verification

5.1 Development Stage

Because of frequent modification during the development stage, Ambarella's chip provides POC (Power on Configuration) that allows the user to enable and disable the secure boot on the board. The RTOS booting flow for BST and BLD mentioned in Section 2.2 will then be performed based on the POC_6 setting.

Ambarella Confidential
For Wakra Lang only
2.2 (7/05/2025)

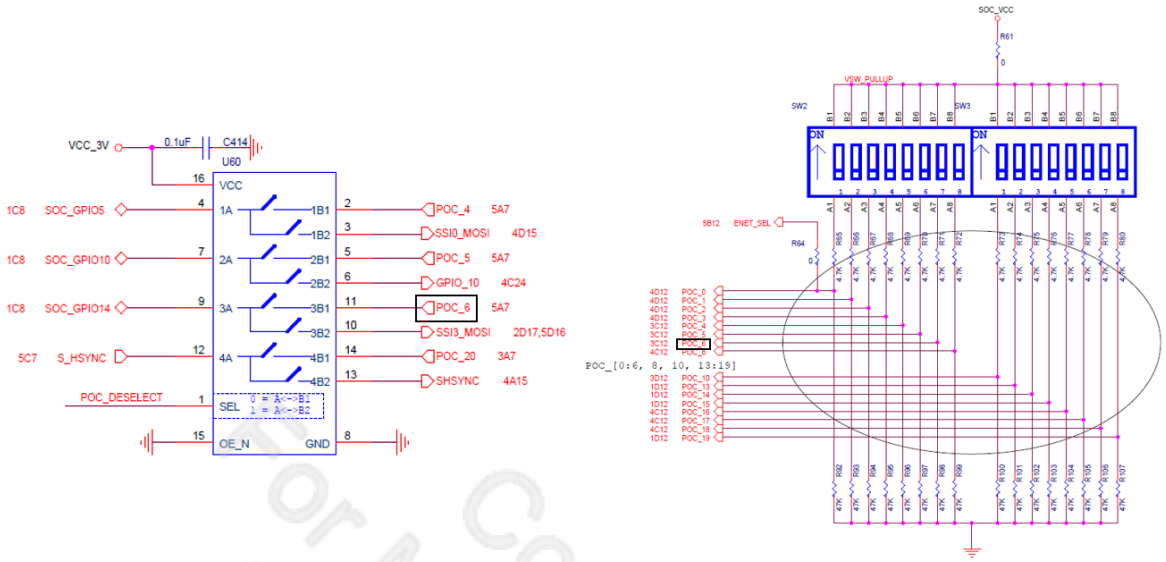


Figure 20 *POC_6 Setting on CV25 BUB V300 Board for Example.*

To verify ATF, IPL (QNX only) and SYS binary, which are controlled by software, please follow Section 4.2 to compile the corresponding firmware to enable.

5.2 Mass Production (MP) Stage

In the MP stage, product will not have the POC_6 to control the booting option. The user must follow Section 3.3, OTP, “Permanently Enable Secure Boot”, using the commands to enable the secure boot flow in the kernel. After rebooting, the system will verify BST and BLD with secure ROM for every boot-up. Same as the development stage, to verify ATF, IPL and SYS binary follow chapter 4.2 to compile the corresponding firmware to enable. After enabling “Permanently Enable Secure Boot”, POC_6 cannot be adjusted anymore, which means the system will always verify BST/BLD with secure ROM.

```
a:\> atf permanentlyenablesecureboot
```

5.3 Return Merchandise Authorization (RMA) Stage

When product is shipped with secure boot enabled, the development function, like JTAG and USB boot, are disabled. This could effectively prevent the internal firmware of the product from being hacked. When the product is sent back for issue investigation, JTAG connection is still required for debugging facility. The JTAG enable register could only be accessed by the secure world software. So, the customers can either have a secure debug application mode to run under secure OS, or define a proprietary authentication mechanism to authorize non-secure software application to re-enable the JTAG by calling API AmbaSYS_JtagOn in the secure OS.

The JTAG enabled by the register with secure boot enabled is not permanent, it only takes effect during power-on condition. Once the chip loses power and is powered on with secure boot enabled again, the JTAG remains disabled.

Confidential
For Mekra Lang only
2.2 (1105202 5)

6 Revision History

Our goal is to provide our customers with the highest-quality documentation possible, and to continuously improve our publications to ensure that your experience with Ambarella's products is a positive one. If you have any questions or comments regarding this document, please contact the Technical Writing team at docs@ambarella.com. Your feedback is welcomed and appreciated.
NOTE: Page/chapter numbers for previous drafts may differ from those in the current version.

Version	Date	Comments
1.0	30 December 2019	Initial version
1.1	24 September 2020	Added notes for public key revoke
1.2	26 October 2020	Added Chapter 4 and Section 3.2 Secure Storage Test Command
1.2	12 December 2020	Added Chapter 5 and Section 4.5
2.0	02 September 2021	Added Query OTP Setting API
2.1	03 August 2022	Added CV5x
2.2	05 July 2023	Added CV2xfs boot diagrams

Confidential For Meka Lang only 2.2 (11052025)